

파이썬 기초와 CSV 데이터 다루기 보강

각 단원에서 “왜 배우는지 → 코드가 어떻게 동작하는지 → 결과를 어떻게 해석하는지 → 어떤 오류가 자주 나는지”를 초보자 기준으로 더 자세히 설명합니다.

메인

Chapter 1

세미프로젝트

1. Chapter 1을 배우는 이유

데이터 분석 수업에서 파이썬 문법을 배우는 이유는 문법 자체를 암기하기 위해서가 아닙니다. 엑셀이나 CSV에 들어 있는 데이터를 코드로 읽고, 필요한 값을 계산하고, 조건에 맞는 데이터만 골라내기 위해 최소한의 문법이 필요합니다.

입문자는 처음부터 모든 문법을 완벽히 외우려고 하면 쉽게 지칩니다. 이 장에서는 “데이터를 다루기 위해 꼭 필요한 문법”만 먼저 익힙니다. 변수는 값을 저장하기 위해, 리스트와 튜플은 여러 값을 묶기 위해, 조건문과 반복문은 많은 데이터를 자동으로 처리하기 위해 사용합니다.

학습 관점

문법 암기보다 데이터 처리 상황과 연결합니다.

실습 관점

코드를 직접 실행하고 출력 결과를 확인합니다.

오류 관점

오류 메시지를 읽고 원인을 추정하는 습관을 만듭니다.

2. 주피터 노트북 학습법

주피터 노트북은 초보자에게 좋은 실습 환경입니다. 코드를 한 줄씩 실행해 볼 수 있고, 코드 사이에 설명을 마크다운으로 남길 수 있기 때문입니다. 수업에서는 한 번에 긴 코드를 작성하기보다 작은 셀 단위로 나누어 실행하는 방식이 좋습니다.

권장 셀 구성

1. 마크다운 셀에 이번 셀에서 확인할 내용을 적습니다.
2. 코드 셀에 3~10줄 정도의 작은 코드를 작성합니다.
3. 실행 결과를 확인합니다.
4. 결과가 예상과 다르면 변수명, 괄호, 따옴표, 들여쓰기를 점검합니다.

```
# 좋은 실습 흐름 예시
# 1. 데이터를 만든다.
scores = [80, 90, 75]

# 2. 평균을 계산한다.
average = sum(scores) / len(scores)

# 3. 결과를 출력한다.
print("평균 점수:", average)
```

초보자 팁: 오류가 나면 코드 전체를 지우지 말고, 가장 마지막에 수정한 줄부터 확인하세요. 대부분의 초보자 오류는 오타, 괄호, 따옴표, 들여쓰기에서 발생합니다.

3. 파이썬 기본 문법 1 보강

사칙연산, 변수, 자료형, 문자열, 리스트, 튜플은 이후 모든 실습의 바탕입니다. 특히 데이터 분석에서는 컬럼 값을 계산하고, 파일명을 변수에 저장하고, 여러 컬럼명을 리스트로 관리하는 일이 많습니다.

변수를 쓰는 실무적 이유

변수를 사용하면 값에 이름을 붙일 수 있습니다. 예를 들어 기준 점수 80을 여러 곳에서 직접 쓰면 나중에 기준이 바뀌었을 때 모든 코드를 찾아 수정해야 합니다. 하지만 `pass_score = 80` 처럼 변수로 저장하면 한 곳만 바꾸면 됩니다.

```
pass_score = 80
score = 85

if score >= pass_score:
    print("합격")
```

리스트와 튜플 구분

구분	사용 상황	예시
리스트	값이 바뀔 수 있는 데이터 묶음	점수 목록, 판매 수량
튜플	잘 바뀌지 않는 고정 값 묶음	메뉴 이름, 요일 이름

실습에서는 메뉴 이름은 튜플, 가격과 판매 수량은 리스트로 저장해 보면 두 자료형의 차이를 자연스럽게 이해할 수 있습니다.

4. 파이썬 기본 문법 2 보강

조건문과 반복문은 데이터 분석 자동화의 핵심입니다. 데이터가 3개일 때는 직접 계산해도 되지만, 데이터가 300개 또는 30,000개가 되면 반복문과 조건문이 필요합니다.

조건문을 데이터 분석에 연결하기

조건문은 특정 기준을 만족하는지 판단합니다. 예를 들어 점수가 80점 이상인지, 매출이 40,000원 이상인지, 파일명이 csv로 끝나는지 판단할 때 사용합니다.

```
sales = 54000

if sales >= 40000:
    print("우수 판매 메뉴")
else:
    print("일반 판매 메뉴")
```

반복문을 쓰는 이유

반복문은 같은 작업을 여러 데이터에 반복 적용합니다. 메뉴 3개의 매출을 계산할 때도 반복문을 쓰면 코드가 간결해지고, 메뉴가 100개로 늘어나도 같은 구조로 처리할 수 있습니다.

```
menus = ["아메리카노", "카페라떼", "샌드위치"]
prices = [4500, 5000, 6500]
counts = [12, 8, 5]

for i, menu in enumerate(menus):
    sales = prices[i] * counts[i]
    print(menu, sales)
```

5. NumPy 보강

NumPy는 숫자 계산을 편하게 하기 위한 도구입니다. 리스트로도 계산할 수 있지만, NumPy 배열을 사용하면 여러 숫자에 같은 계산을 한 번에 적용할 수 있습니다. 머신러닝 모델도 내부적으로는 많은 숫자 배열을 계산합니다.

리스트와 NumPy 배열의 차이

```
import numpy as np

scores_list = [70, 80, 90]
scores_array = np.array([70, 80, 90])

# 리스트는 이런 계산이 바로 되지 않습니다.
```

```
# print(scores_list + 5)

# NumPy 배열은 전체 원소에 한 번에 계산이 적용됩니다.
print(scores_array + 5)
```

이 차이는 나중에 여러 컬럼 값을 한 번에 변환하거나, 모델 입력 데이터를 계산할 때 매우 중요합니다.

조건 필터링 이해

조건 필터링은 "조건을 만족하는 값만 선택하는 방법"입니다. pandas의 행 필터링도 이와 같은 방식으로 이해할 수 있습니다.

```
scores = np.array([70, 85, 90, 60, 95])
mask = scores >= 80
print(mask)
print(scores[mask])
```

6. 파일 입출력과 CSV 보강

데이터 분석은 대부분 외부 파일에서 데이터를 읽어오면서 시작합니다. 파일 경로를 정확히 이해하지 못하면 `FileNotFoundError` 가 자주 발생합니다. 따라서 파일명, 폴더 위치, 상대 경로를 먼저 확인하는 습관이 중요합니다.

CSV를 배워야 하는 이유

CSV는 가장 단순하고 널리 쓰이는 데이터 교환 형식입니다. 엑셀에서도 열 수 있고, 파이썬에서도 쉽게 읽을 수 있습니다. 실무에서 시스템 간 데이터를 주고받을 때 CSV 형식이 자주 사용됩니다.

자주 나는 오류

오류	원인	해결 방법
----	----	-------

FileNotFoundError

파일 경로가 잘못됨

현재 작업 폴더와 파일 위치 확인

UnicodeDecodeError

인코딩 문제

`encoding="utf-8"` 또는 `cp949` 확인

KeyError

컬럼명 오타

`df.columns` 로 실제 컬럼명 확인

7. 실습 확장 과제

기존 단원별 실습을 마친 뒤에는 아래 순서로 난이도를 조금 높여보는 것이 좋습니다.

1. 예시 데이터의 값을 직접 바꿔서 결과가 어떻게 달라지는지 확인합니다.
2. 기준값을 변수로 분리합니다. 예: `pass_score` , `sales_threshold`
3. 출력 결과를 더 읽기 좋은 문장으로 바꿉니다.
4. 조건을 하나 더 추가합니다. 예: 매출 40,000원 이상이면서 판매 수량 5개 이상
5. 결과를 CSV 또는 txt 파일로 저장합니다.

Chapter 1 핵심 정리: 파이썬 기초 문법은 데이터 분석을 위한 도구입니다. 변수, 리스트, 조건문, 반복문, 파일 입출력, CSV 처리를 연결해서 “작은 자동화 프로그램”을 만들 수 있으면 Chapter 1 목표를 달성한 것입니다.

[← Chapter 1 목록](#)

[세미프로젝트로 이동 →](#)